

# Redes de Computadores II

## Ciência da Computação

Robson Costa, Dr.  
`robson.costa@ifsc.edu.br`

Instituto Federal de Santa Catarina  
Campus Lages

# Sumário

## Unidade 2 – Roteamento

2.1 - Entrega direta e indireta

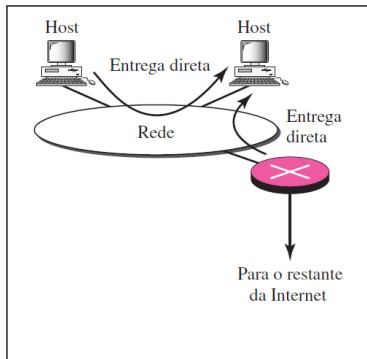
2.2 - Encaminhamento

2.3 - Protocolos de roteamento

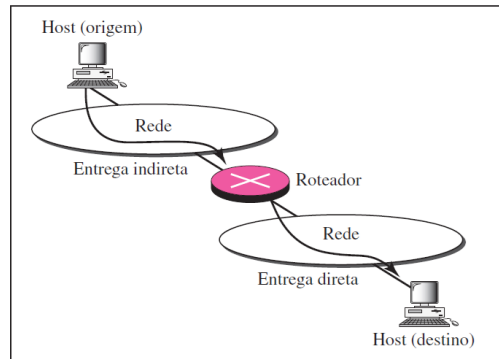
# Entrega direta e indireta

- **Entrega direta:** o destino final do pacote é um *host* conectado à mesma rede física que a do entregador;
  - Para determinar se a entrega é direta, se extrai o endereço de rede do destino (usando a máscara) e se compara o endereço obtido com os endereços das redes às quais o nó está conectado; se for encontrada alguma coincidência, a entrega é direta;
- **Entrega indireta:** Quando o *host* de destino não se encontra na mesma rede do entregador;
  - Neste caso o pacote vai de roteador em roteador até atingir aquele conectado à mesma rede física de seu destino final;

# Entrega direta e indireta



a. Entrega direta



b. Entrega direta e indireta

# Encaminhamento

- O **encaminhamento** significa colocar o pacote na rota para seu destino;
- Requer que um *host* ou um roteador tenha uma tabela de roteamento;
- Quando um *host* tiver um pacote a ser enviado ou quando um roteador tiver recebido um pacote a ser encaminhado, ele busca essa tabela para encontrar a rota para o destino final;
- **PROBLEMA:** essa solução simples é impossível hoje em dia, em uma infraestrutura como a Internet, pois o número de entradas necessárias na tabela de roteamento tornaria ineficientes as pesquisas em tabelas;
- Para resolver isso, várias técnicas podem ser utilizadas para manter administrável o tamanho da tabela de roteamento;

# Encaminhamento - Técnicas de encaminhamento

- **Método do próximo salto:** permite a redução no tamanho da tabela de roteamento onde ela armazena apenas o endereço do próximo salto, em vez das informações sobre a rota completa;
- As entradas de uma tabela de roteamento devem ser consistentes entre si;

a. Tabelas de roteamento baseadas em rotas

| Destino | Rota           |
|---------|----------------|
| Host B  | R1, R2, host B |

Tabela de roteamento para A

| Destino | Rota       |
|---------|------------|
| Host B  | R2, host B |

Tabela de roteamento para R1

| Destino | Rota   |
|---------|--------|
| Host B  | Host B |

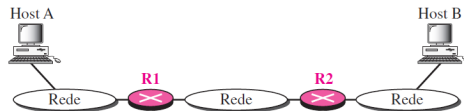
Tabela de roteamento para R2

b. Tabelas de roteamento baseadas no próximo salto

| Destino | Próximo salto |
|---------|---------------|
| Host B  | R1            |

| Destino | Próximo salto |
|---------|---------------|
| Host B  | R2            |

| Destino | Próximo salto |
|---------|---------------|
| Host B  | ---           |



# Encaminhamento - Técnicas de encaminhamento

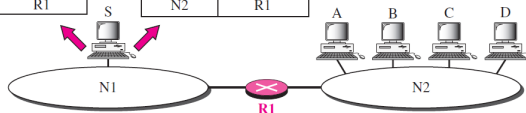
- **Método da rede específica:** também permite a redução do tamanho da tabela de roteamento;
- Neste caso, em vez de ter uma entrada para cada *host* de destino conectado à mesma rede física (método do *host* específico), temos apenas uma entrada que define o endereço da rede de destino em si;
- Neste caso, todos os *hosts* conectados à mesma rede são tratados como uma única entidade;

Tabela de roteamento para o host S baseada no método de host específico

| Destino | Próximo salto |
|---------|---------------|
| A       | R1            |
| B       | R1            |
| C       | R1            |
| D       | R1            |

Tabela de roteamento para o host S baseada no método de rede específica

| Destino | Próximo salto |
|---------|---------------|
| N2      | R1            |

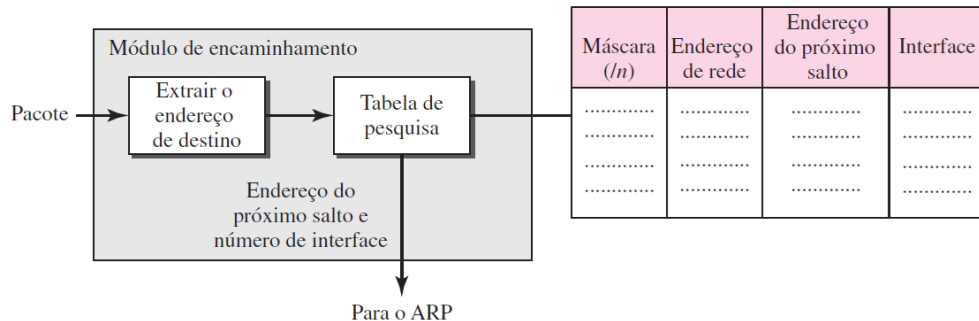


# Encaminhamento - Processo de encaminhamento

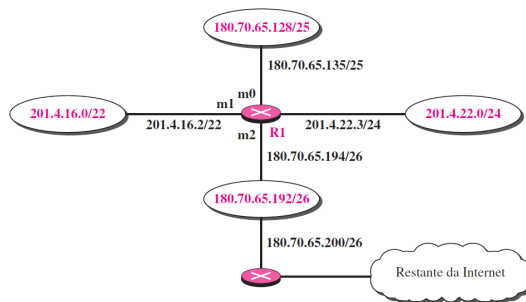
- Suponhamos que os *hosts* e roteadores usem endereçamento sem classes (o endereçamento com classes pode ser tratado como um caso especial do endereçamento sem classes);
- Neste caso, a tabela de roteamento precisa ter uma linha de informações para cada bloco envolvido;
- A tabela precisa ser pesquisada tomando-se como base o endereço de rede (o primeiro endereço no bloco);
- O endereço de destino no pacote não dá nenhuma pista sobre o endereço de rede;
- Para resolver esse problema, precisamos incluir a máscara no formato CIDR (*/n*) para o bloco correspondente;



# Encaminhamento - Processo de encaminhamento



# Encaminhamento - Processo de encaminhamento



| Máscara  | Endereço de Rede | Próximo salto | Interface |
|----------|------------------|---------------|-----------|
| /26      | 180.70.65.192    | —             | m2        |
| /25      | 180.70.65.128    | —             | m0        |
| /24      | 201.4.22.0       | —             | m3        |
| /22      | 201.4.16.0       | ....          | m1        |
| Qualquer | Qualquer         | 180.70.65.200 | m2        |

# Encaminhamento - Processo de encaminhamento

- **Exemplo 1:** Chegada de um pacote em R1 com o endereço de destino 180.70.65.140.
  - ① A primeira máscara (/26) é aplicada ao endereço de destino; o resultado é o endereço de rede 180.70.65.128, que não coincide com o endereço de rede desta entrada na tabela;
  - ② A segunda máscara (/25) é aplicada ao endereço de destino; o resultado é o endereço de rede 180.70.65.128, que coincide com o endereço de rede desta entrada na tabela;
    - O endereço do próximo salto (nesse caso, o endereço de destino do pacote) e o número de interface (m0) são passados ao ARP para processamento adicional;

# Encaminhamento - Processo de encaminhamento

- **Exemplo 2:** Chegada de um pacote em R1 com o endereço de destino 201.4.22.35.
  - 1 A primeira máscara (/26) é aplicada ao endereço de destino; o resultado é o endereço de rede 201.4.22.0, que não coincide com o endereço de rede desta entrada na tabela;
  - 2 A segunda máscara (/25) é aplicada ao endereço de destino; o resultado é o endereço de rede 201.4.22.0, que não coincide com o endereço de rede desta entrada na tabela;
  - 3 A terceira máscara (/24) é aplicada ao endereço de destino; o resultado é o endereço de rede 201.4.22.0, que coincide com o endereço de rede desta entrada na tabela;
    - O endereço de destino do pacote e o número de interface (m3) são passados ao ARP;

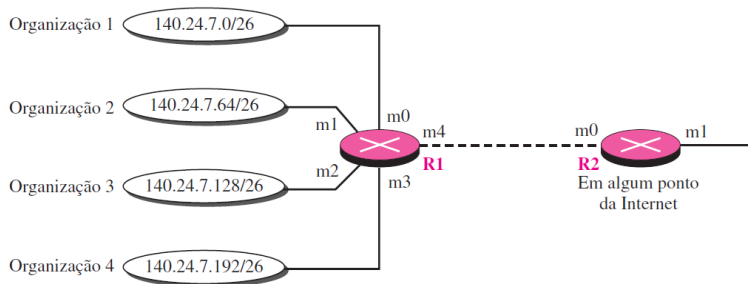
# Encaminhamento - Processo de encaminhamento

- **Exemplo 3:** Chegada de um pacote em R1 com o endereço de destino 18.24.32.78.
  - 1 Todas as máscaras serão aplicadas e nenhum endereço de rede correspondente será encontrado;
  - 2 Ao chegar ao final da tabela, o módulo passa o endereço do próximo nó, 180.70.65.200, e o número de interface (m2) para o ARP;
    - Provavelmente este é um pacote de saída que precisa ser enviado, por meio do roteador-padrão, para algum outro lugar na Internet;

# Encaminhamento - Processo de encaminhamento

- Quando usamos endereçamento sem classes (CIDR), é provável que o número de entradas da tabela de roteamento vá aumentar. uma vez que ele irá dividir todo o espaço de endereços em blocos gerenciáveis;
- O tamanho maior da tabela resulta em um aumento do tempo necessário para pesquisar a tabela;
- Para atenuar este problema, foi desenvolvido o conceito de **agregação de endereços** onde os blocos de endereços para as organizações conectadas à um roteador são agregados em um único bloco maior;

# Encaminhamento - Processo de encaminhamento



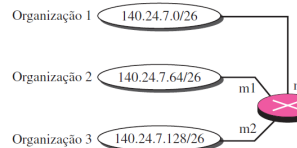
| Máscara | Endereço de rede | Endereço do próximo salto | Interface |
|---------|------------------|---------------------------|-----------|
| /26     | 140.24.7.0       | -----                     | m0        |
| /26     | 140.24.7.64      | -----                     | m1        |
| /26     | 140.24.7.128     | -----                     | m2        |
| /26     | 140.24.7.192     | -----                     | m3        |
| /0      | 0.0.0.0          | Padrão                    | m4        |

Tabela de roteamento para R1

| Máscara | Endereço de rede | Endereço do próximo salto | Interface |
|---------|------------------|---------------------------|-----------|
| /24     | 140.24.7.0       | -----                     | m0        |
| /0      | 0.0.0.0          | Padrão                    | m1        |

Tabela de roteamento para R2

# Encaminhamento - Processo de encaminhamento

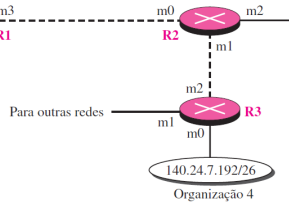


| Máscara | Endereço de rede | Endereço do próximo salto | Interface |
|---------|------------------|---------------------------|-----------|
| /26     | 140.24.7.0       | -----                     | m0        |
| /26     | 140.24.7.64      | -----                     | m1        |
| /26     | 140.24.7.128     | -----                     | m2        |
| /0      | 0.0.0.0          | Padrão                    | m3        |

Tabela de roteamento para R1

Tabela de roteamento para R2

| Máscara | Endereço de rede | Endereço do próximo salto | Interface |
|---------|------------------|---------------------------|-----------|
| /26     | 140.24.7.192     | -----                     | m1        |
| /24     | 140.24.7.0       | -----                     | m0        |
| /??     | ???????          | ?????????                 | m1        |
| /0      | 0.0.0.0          | Padrão                    | m2        |



| Máscara | Endereço de rede | Endereço do próximo salto | Interface |
|---------|------------------|---------------------------|-----------|
| /26     | 140.24.7.192     | -----                     | m0        |
| /??     | ???????          | ?????????                 | m1        |
| /0      | 0.0.0.0          | Padrão                    | m2        |

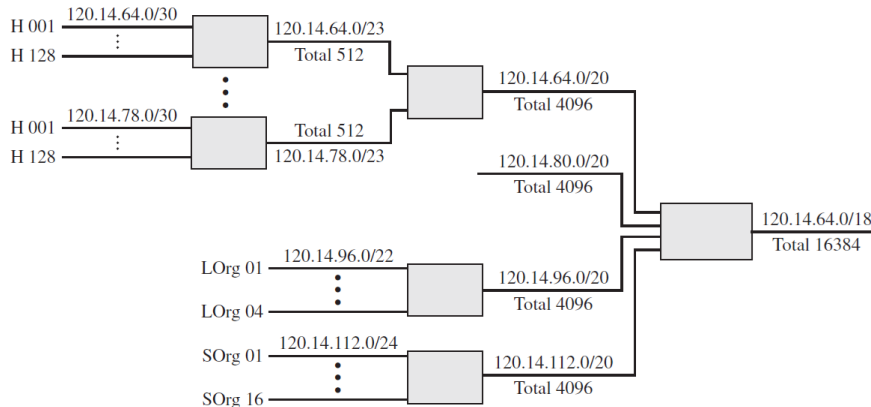
Tabela de roteamento para R3



# Encaminhamento - Processo de encaminhamento

- Para resolver o problema de tabelas de roteamento enormes, podemos criar nelas um sentido de hierarquia, o chamado **roteamento hierárquico**;
- Atualmente a Internet é dividida em ISPs (*Internet Service Provider*) nacionais e internacionais;
- Os ISPs nacionais se dividem em ISPs regionais e estes em ISPs locais;

# Encaminhamento - Processo de encaminhamento



# Encaminhamento - Tabela de roteamento

- Um *host* ou um roteador apresenta uma tabela de roteamento com uma entrada para cada destino ou para uma combinação de destinos, a fim de direcionar pacotes IP;
- **Tabela estática:** contém informações introduzidas manualmente pelo administrador para cada destino, não podendo ser atualizada automaticamente quando existe alguma mudança na Internet;
- **Tabela dinâmica:** atualizada periodicamente usando-se um dos protocolos de roteamento dinâmico como o RIP, OSPF ou BGP; toda vez que houver uma mudança na Internet (ex.: o desligamento de um roteador ou a quebra de um enlace) os protocolos de roteamento dinâmico atualizam automaticamente todas as tabelas nos roteadores (e, eventualmente, no *host*);

# Protocolos de roteamento

- Os protocolos de roteamento foram criados em resposta à demanda por tabelas de roteamento dinâmicas;
- Um protocolo de roteamento é uma combinação de regras e procedimentos que possibilita aos roteadores na Internet informarem as mudanças entre si;
- Ele permite que os roteadores compartilhem tudo o que souberem em relação à Internet ou à sua vizinhança;
- O compartilhamento de informações permite que um roteador em Lages saiba sobre a falha de uma rede em Florianópolis;
- Os protocolos de roteamento também incluem procedimentos para combinar informações recebidas de outros roteadores;

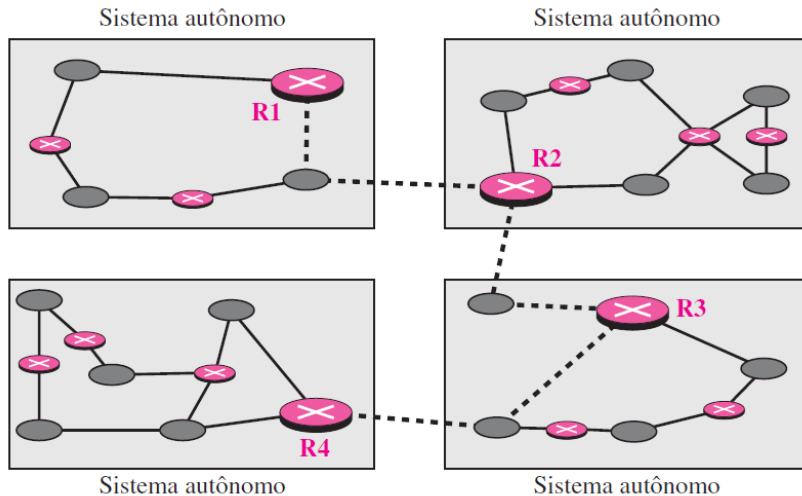
# Protocolos de roteamento

- Um roteador recebe um pacote de uma rede e o passa para outra;
- Normalmente, um roteador está conectado à várias redes, sendo assim, ao receber um pacote, para qual rede deve passar o pacote?
- A decisão se baseia na **otimização**: qual dos caminhos disponíveis é o **melhor**? Qual a definição do termo **melhor caminho**?
  - Utiliza-se uma métrica chamada de *custo*, mas isso depende do tipo de protocolo;
  - Alguns protocolos utilizam como referência o número de saltos (*hop*) da origem ao destino, outros atribuem pesos diferentes para cada salto e contabilizam o menor custo;
  - Vale lembrar que em roteamento cada salto é caracterizado por um enlace que conecta dois roteadores;

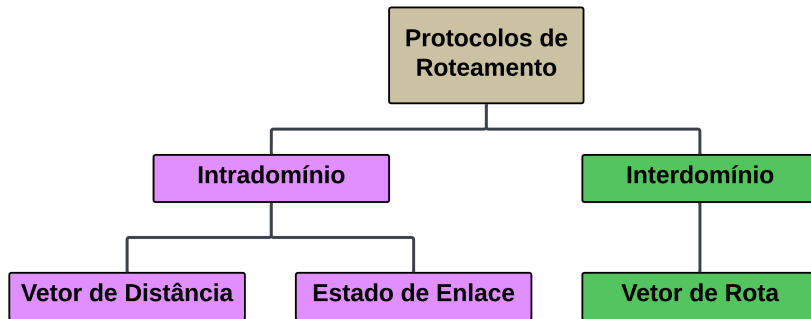
## Protocolos de roteamento - Roteamento interdomínio e intradomínio

- Hoje em dia, a Internet pode ser tão grande que um protocolo de roteamento não é capaz de lidar com a tarefa de atualizar as tabelas de roteamento de todos os roteadores;
- Por essa razão, uma Internet é dividida em **Sistemas Autônomos (AS – *Autonomous System*)** que é um grupo de redes e roteadores sob a regência de uma única administração;
- O roteamento dentro de um sistema autônomo é denominado roteamento **intradomínio**;
- O roteamento entre sistemas autônomos é conhecido como roteamento **interdomínio**;
- Cada sistema autônomo pode escolher um ou mais protocolos de roteamento intradomínio para tratar do roteamento dentro do sistema autônomo;
- Entretanto, apenas um protocolo de roteamento interdomínio trata do roteamento entre sistemas autônomos;

# Protocolos de roteamento - Roteamento interdomínio e intradomínio



# Protocolos de roteamento - Roteamento interdomínio e intradomínio





# Protocolos de roteamento - Roteamento vetor de distância

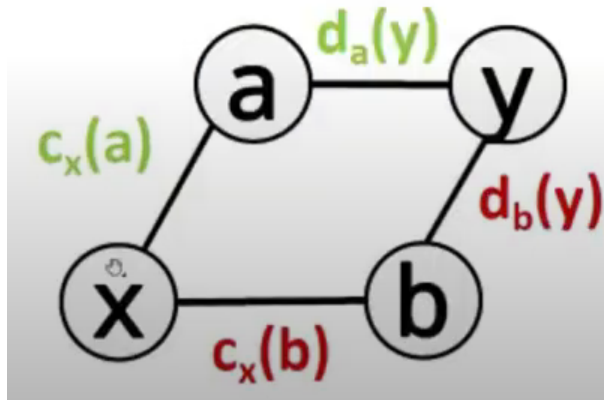
- No roteamento **vetor de distância**, o melhor caminho entre dois nós quaisquer é a rota com distância mínima;
- Nesse protocolo, como o próprio nome implica, cada nó mantém um vetor (tabela) de distâncias mínimas para todos os nós;
- A tabela em cada nó também orienta os pacotes para o nó desejado mostrando a próxima parada na rota (roteamento até o próximo salto);
- Baseado no algoritmo de ***Bellman-Ford***;
  - Utilizado para determinar o menor custo entre origem e destino através de nós intermediários;

# Protocolos de roteamento - Roteamento vetor de distância

- Equação de *Bellman-Ford*:
  - Considere que:
    - $d_x(y)$  é o custo do caminho de menor custo entre X e Y;
    - $c_x(z)$  é o custo entre dois nós X e Z;
    - x é a origem;
    - y é o destino;
    - a e b são nós intermediários;
  - Então o custo do **menor caminho** é dado por:
    - $d_x(y) = \min\{c_x(a) + d_a(y), c_x(b) + d_b(y)\}$

# Protocolos de roteamento - Roteamento vetor de distância

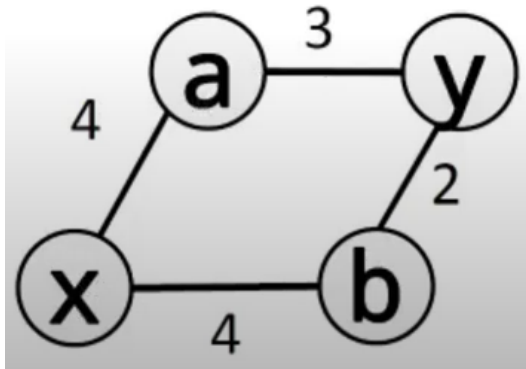
- Equação de *Bellman-Ford*:
  - $d_x(y) = \min\{c_x(a) + d_a(y), c_x(b) + d_b(y)\}$



# Protocolos de roteamento - Roteamento vetor de distância

- Equação de *Bellman-Ford*:

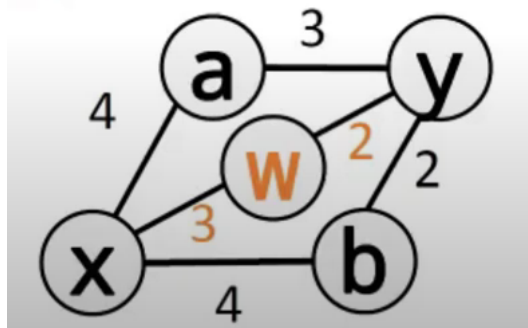
- $d_x(y) = \min\{c_x(a) + d_a(y), c_x(b) + d_b(y)\}$
- $d_x(y) = \min\{4 + 3, 4 + 2\} = 6$



# Protocolos de roteamento - Roteamento vetor de distância

- Equação de *Bellman-Ford*:

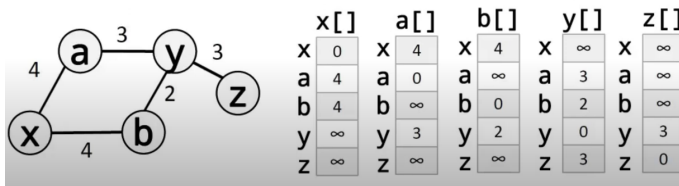
- Quando um novo nós é inserido, o custo mínimo é atualizado e esse valor é mantido como o novo  $d_x(y)$ ;
- $d_x(y) = \min\{d_x(y), c_x(w) + d_w(y)\}$
- $d_x(y) = \min\{6, 3 + 2\} = 5$



# Protocolos de roteamento - Roteamento vetor de distância

## Inicialização

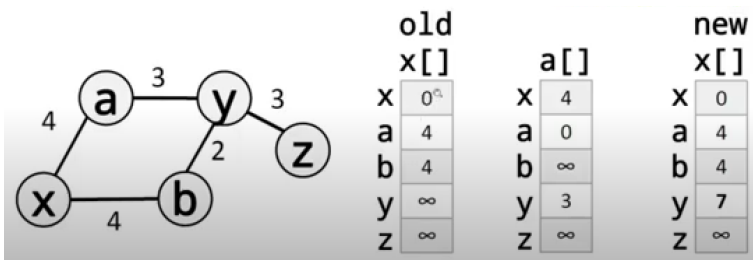
- Cada nó tem conhecimento apenas da distância entre ele e seus vizinhos imediatos (aqueles conectados diretamente a ele);
- A distância para qualquer entrada que não seja um vizinho é marcada como infinita (inatingível);



# Protocolos de roteamento - Roteamento vetor de distância

## Atualização

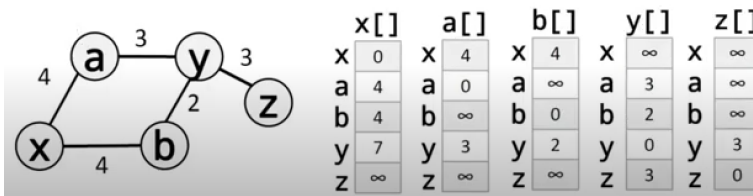
- Caso o nó  $x$  receba o vetor do nó  $a$  ele atualiza os seus valores internos:
- $\text{new } x[] = \min\{x[], 4 + a[]\}$



# Protocolos de roteamento - Roteamento vetor de distância

## Atualização

- Após alguns compartilhamentos, espera-se que todos os nós conheçam o melhor caminho até outros nós da rede;

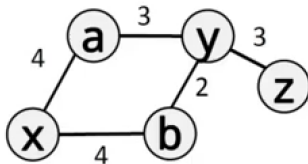




# Protocolos de roteamento - Roteamento vetor de distância

## Roteamento

- Através do cálculo dos vetores de distância é possível montar **tabelas de roteamento**;



|   | x[] |
|---|-----|
| x | 0   |
| a | 4   |
| b | 4   |
| y | 6   |
| z | 9   |

até via custo

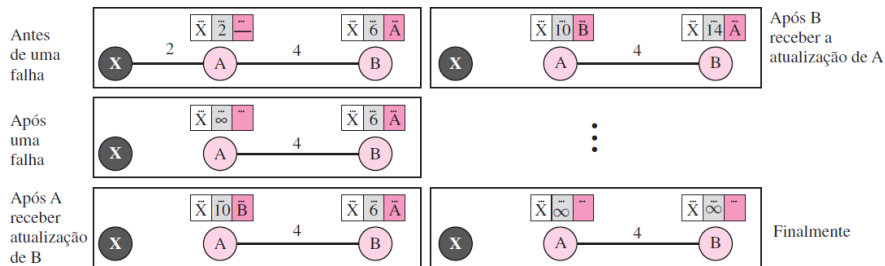
|   |   |   |
|---|---|---|
| x | - | 0 |
| a | - | 4 |
| b | - | 4 |
| y | b | 6 |
| z | b | 9 |

tabela de roteamento de x<sub>21</sub>

# Protocolos de roteamento - Roteamento vetor de distância

## Instabilidade de *Loop* de Dois Nós

- Ocorre quando a atualização acerca da falha de uma rota em um nó acaba sendo sobreposta pela atualização de um nó vizinho;



# Protocolos de roteamento - Roteamento vetor de distância

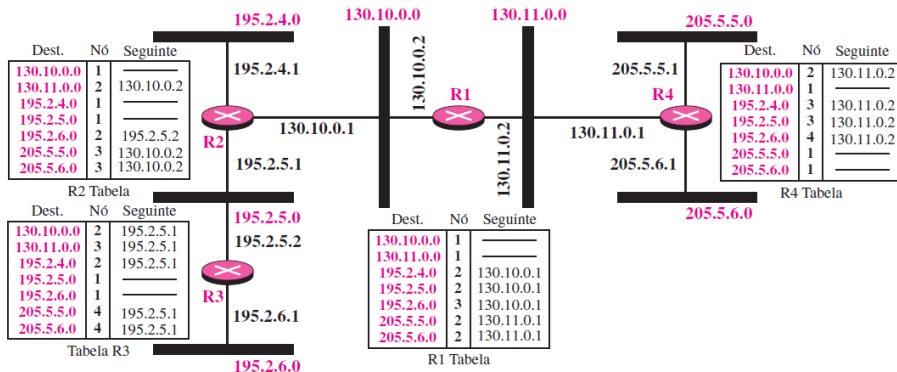
## RIP (*Routing Information Protocol*)

- Lançado originalmente (RIPv1) em 1988:
  - Envia uma atualização da sua tabela de roteamento a cada 30 segundos;
  - Faz *broadcast* de um datagrama UDP enviado pela porta 520;
- Em 1994 foi lançado o RIPv2:
  - Adicionou compatibilidade com CIDR (*Classless Inter-Domain Routing*);
  - Trocou o compartilhamento via *broadcast* por *multicast* pelo endereço 224.0.0.9;
- Em 1997 foi adicionado a autenticação com o MD5 assim como foi criada a versão RIPv2, um extensão do RIPv2 com suporte ao IPv6;

# Protocolos de roteamento - Roteamento vetor de distância

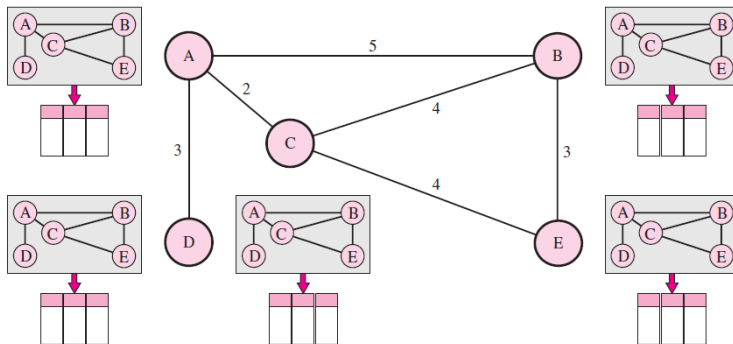
## RIP (*Routing Information Protocol*)

- Protocolo de roteamento **intradomínio** usado dentro de um sistema autônomo;
- Realiza o roteamento entre redes utilizando a **contagem de nós** como métrica;
- O infinito é definido em 16, ou seja, funciona para distâncias máximas de 15 nós;



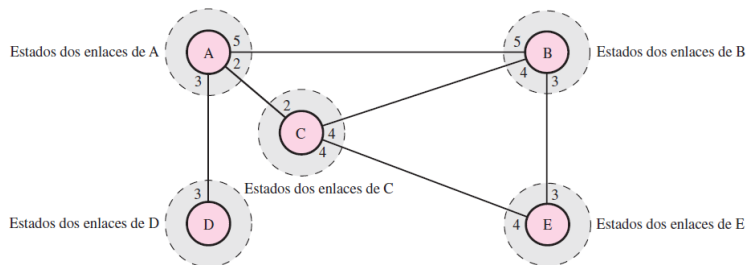
# Protocolos de roteamento - Roteamento estado de enlaces

- Assume que se cada nó no domínio tiver toda a topologia do domínio (a lista dos nós e enlaces, como eles são interligados incluindo o tipo, custo (métrica) e condição dos enlaces ativos ou inativos) o nó poderá usar o **algoritmo de Dijkstra** para construir a tabela de roteamento;



# Protocolos de roteamento - Roteamento estado de enlaces

- Embora este modelo assuma que cada nó no domínio possui conhecimento da topologia completa, assume também que cada nó possui conhecimento parcial do estado dos enlaces (apenas os enlaces diretamente ligados ao nós);



# Protocolos de roteamento - Roteamento estado de enlaces

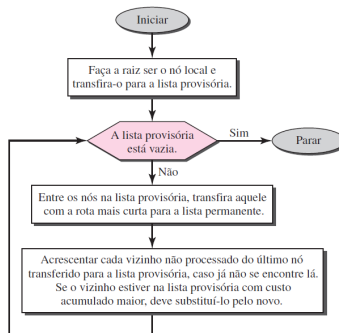
## Inicialização

- Quatro etapas são necessárias para a inicialização do algoritmo:
  - ① Criação dos estados dos enlaces por cada nó **LSP (*Link State Packet*)**;
    - Envia o ID do nó, a sua lista de enlaces, um número de sequência e a idade;
  - ② Disseminação de LSPs para cada um dos demais roteadores, a chamada inundação, de uma forma eficiente e confiável;
    - Envia para todos os nós da rede (não apenas aos seus vizinhos);
    - Ao receber um LSP o nó verifica qual o mais novo (pelo número de sequência); se o LSP recebido foi mais novo ele atualiza os valores internos e dissemina as suas novas informações para todas as interfaces menos aquele ao qual recebeu o LSP, caso contrário, descarta o LSP;
  - ③ Formação de uma árvore de rota mais curta para cada nó utilizando Dijkstra;
  - ④ Cálculo de uma tabela de roteamento com base na árvore de rota mais curta;

# Protocolos de roteamento - Roteamento estado de enlaces

## Algoritmo de Dijkstra

- O algoritmo de **Dijkstra** cria uma árvore de rota mais curta a partir de um grafo, cuja o objetivo é definir o melhor caminho entre o nó origem e os destinos existentes na tabela de roteamento;





# Protocolos de roteamento - Roteamento estado de enlaces

## Algoritmo de Dijkstra

```
1  function Dijkstra(Graph, source):  
2  
3      for each vertex v in Graph.Vertices:  
4          dist[v] ← INFINITY  
5          prev[v] ← UNDEFINED  
6          add v to Q  
7      dist[source] ← 0  
8  
9      while Q is not empty:  
10         u ← vertex in Q with minimum dist[u]  
11         remove u from Q  
12  
13         for each neighbor v of u still in Q:  
14             alt ← dist[u] + Graph.Edges(u, v)  
15             if alt < dist[v]:  
16                 dist[v] ← alt  
17                 prev[v] ← u  
18  
19      return dist[], prev[]
```

# Protocolos de roteamento - Roteamento estado de enlaces

## OSPF (*Open Shortest Path First*)

- Descrito inicialmente em 1989 pela RFC 1131;
- Baseado no algoritmo **SPF (*Shortest Path First*)**, o algoritmo de Dijkstra;
- Em 1991 foi lançada a versão 2 descrita na RFC 1247 a qual sofreu diversas atualizações até o lançamento da versão 3 em 2008 descrita na RFC 5340 (suporte ao IPv6);
- A sua última atualização foi descrita na RFC 9454 em 2023;
- Diferente de outros protocolos de roteamento, o OSPF não utiliza protocolos de transporte (UDP ou TCP) para compartilhar os seus dados, ao invés disso envia os dados diretamente no pacote IP;
- O compartilhamento de tabelas é realizado em intervalos de 10 segundos em redes Ethernet;

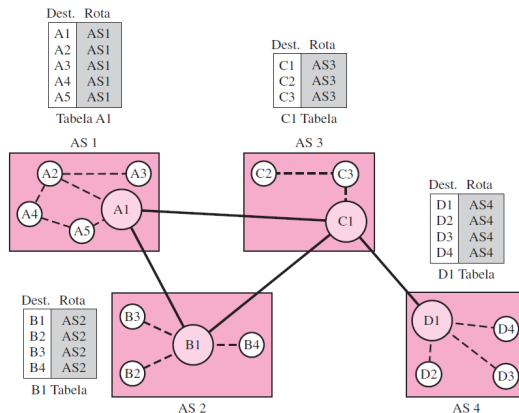
## Protocolos de roteamento - Roteamento vetor de rota

- Hoje em dia, a Internet pode ser tão grande que um protocolo de roteamento não é capaz de lidar com a tarefa de atualizar as tabelas de roteamento de todos os roteadores;
- Neste contexto, algoritmos de roteamento de **vetor de rota** são utilizados para o roteamento de pacotes interdomínio;
- O seu princípio é similar àquele do roteamento vetor distância;
- Assume-se a existência de apenas um nó em cada sistema autônomo que atua em nome do sistema inteiro, o qual é chamado de **nó alto-falante**;
- Ele cria a tabela de roteamento e o anuncia exclusivamente para os nós alto-falantes nos ASs vizinhos;
- A diferença para o vetor de distância é que um nó alto-falante anuncia a rota, não a métrica dos nós, em seu sistema autônomo ou outros sistemas autônomos;

# Protocolos de roteamento - Roteamento vetor de rota

## Inicialização

- No início, cada nó alto-falante sabe apenas da possibilidade de alcançar os nós internos ao seu sistema autônomo;



# Protocolos de roteamento - Roteamento vetor de rota

## Compartilhamento e Atualização

- Um nó alto-falante compartilha a sua tabela de rotas com seus vizinhos diretos;
- Quando um nó alto-falante recebe uma tabela de um vizinho, ele atualiza sua própria tabela acrescentando os nós que não se encontram em sua tabela de roteamento e acrescentando seu próprio sistema autônomo e o sistema autônomo que enviou a tabela;
- Após alguns instantes cada alto-falante tem uma tabela e sabe como atingir cada nó nos demais ASs;

| Dest. | Rota        |
|-------|-------------|
| A1    | AS1         |
| ...   | ...         |
| A5    | AS1         |
| B1    | AS1-AS2     |
| ...   | ...         |
| B4    | AS1-AS2     |
| C1    | AS1-AS3     |
| ...   | ...         |
| C3    | AS1-AS3     |
| D1    | AS1-AS2-AS4 |
| ...   | ...         |
| D4    | AS1-AS2-AS4 |

A1 Tabela

| Dest. | Rota        |
|-------|-------------|
| A1    | AS2-AS1     |
| ...   | ...         |
| A5    | AS2-AS1     |
| B1    | AS2         |
| ...   | ...         |
| B4    | AS2         |
| C1    | AS2-AS3     |
| ...   | ...         |
| C3    | AS2-AS3     |
| D1    | AS2-AS3-AS4 |
| ...   | ...         |
| D4    | AS2-AS3-AS4 |

Tabela B1

| Dest. | Rota    |
|-------|---------|
| A1    | AS3-AS1 |
| ...   | ...     |
| A5    | AS3-AS1 |
| B1    | AS3-AS2 |
| ...   | ...     |
| B4    | AS3-AS2 |
| C1    | AS3     |
| ...   | ...     |
| C3    | AS3     |
| D1    | AS3-AS4 |
| ...   | ...     |
| D4    | AS3-AS4 |

C1 Tabela

| Dest. | Rota        |
|-------|-------------|
| A1    | AS4-AS3-AS1 |
| ...   | ...         |
| A5    | AS4-AS3-AS1 |
| B1    | AS4-AS3-AS2 |
| ...   | ...         |
| B4    | AS4-AS3-AS2 |
| C1    | AS4-AS3     |
| ...   | ...         |
| C3    | AS4-AS3     |
| D1    | AS4         |
| ...   | ...         |
| D4    | AS4         |

D1 Tabela

# Protocolos de roteamento - Roteamento vetor de rota

## **BGP (*Boarder Gateway Protocol*)**

- Descrito inicialmente em 1989 pela RFC 1105;
- A sua última versão (BGPv4) foi lançada em 2006 a qual é descrita na RFC 4271 e sofreu a última atualização em 2024, descrita na RFC 9687;
- Utiliza o protocolo TCP na porta 179 para o transporte de dados;
- Não possui uma atualização periódica como os algoritmos intradomínio, ao invés disso divulga as suas tabelas somente quando ocorrem atualizações nas mesmas (baseado em eventos);